



UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

SEDE JALAPA

FACULTAD DE INGENIERIA EN SISTEMAS DE INFORMACION Y

CIENCIAS DE LA COMPUTACION

SABADO MATUTINO

PROYECTO FINAL

CURSO-032-ANALISIS DE SISTEMAS I

ING. MARCO TULIO VALDEZ ALBIZURES

CICLO-SEPTIMO-2026

INTEGRANTES:

JORGE DANILO UCELO ROMAN	0907-23-18008
CHRISTIAN EDUARDO LOPEZ LEMUS	0907-23-15408
KEILY ATALIA LOPEZ HERNANDEZ	0907-23-21893
CRISTIAN EDUARDO CHAMO MORALES	0907-23-11907
DELMY MARIA FAJARDO BORRAYO	0907-23-11346
LUDIN EDUARDO CARRANZA GUERRA	0907-22-5817

Jalapa, 30 de mayo 2026

INTRODUCCION

El presente proyecto final documenta la aplicación práctica de los conocimientos adquiridos en el curso de Análisis de Sistemas I, aplicados directamente al rediseño operativo del Turicentro Hidden Valley. Ante un escenario real de crecimiento habitacional que superó la capacidad de gestión manual del hotel evidenciando deficiencias críticas como la descentralización de datos, ineficiencia en los flujos de mantenimiento y el riesgo constante de sobreventa. A lo largo del ciclo de desarrollo, se pusieron en práctica los fundamentos de la ingeniería de software y la gestión de proyectos mediante metodologías ágiles. Utilizando Linear como eje central, se experimentó de primera mano la dinámica de los marcos de trabajo modernos al segmentar los requerimientos del negocio en issues, priorizar el backlog y mitigar riesgos a través de ciclos semanales de trabajo (sprints) y reuniones de sincronización diarias (dailys).

1. PROBLEMA A RESOLVER

El Turicentro Hidden Valley ha experimentado un crecimiento habitacional y de afluencia turística durante los últimos 3 años. Sin embargo, la gestión operativa actual representa deficiencias críticas debido a la falta de una herramienta de tecnología para gestionar cada una de las actividades que se realizan, como de tomar las reservaciones y lo que incluye.

- 1.1 **DESCENTRALIZACIÓN Y VULNERABILIDAD DE DATOS:** El registro de huéspedes, la disponibilidad de las cabañas y el control de ingresos de los visitantes se realizan de forma manual o utilizando el calendario normal. Esto provoca duplicidad de datos, pérdida de información de alguna reservación que no se guardó, lo cual no existe ningún historial ni estado de cuenta.
- 1.2 **RIESGO DE SOBREVENTA:** Al no contar con un sistema de tecnología que pueda validar la disponibilidad en tiempo real de cada una de las cabañas, el personal administrativo corre el riesgo de asignar una misma cabaña a diferentes grupos de visitantes en las mismas fechas, afectando directamente la reputación y la desconformidad de las personas.
- 1.3 **INEFICIENCIA EN EL MANTENIMIENTO DE CABAÑAS:** Al no existir un estado de ocupación automatizado y centralizado, el equipo de limpieza y mantenimiento no tiene la certeza inmediata de qué cabañas han sido liberadas o cuáles requieren preparación para un nuevo ingreso. Esto provoca pérdida de tiempo, descoordinación con el personal y tiempo de espera para los nuevos huéspedes, afectando la calidad del servicio a los clientes.

2. SOLUCIÓN PROPUESTA

Como respuesta a esta problemática que se vive actualmente, se desarrolló una aplicación web con arquitectura moderna, diseñada específicamente para automatizar, centralizar y optimizar el control de registros de cabañas para dar un mejor servicio.

Esta solución se sostiene sobre los siguientes pilares fundamentales.

- 2.1 El sistema está dividido en módulos independientes y especializados, lo que garantiza que el fallo de un componente, por ejemplo, el módulo de servicios no afecte la disponibilidad del núcleo del negocio como el registro de visitantes o reservas.
- 2.2 Toda la infraestructura está contenerizada mediante Docker, facilitando un despliegue rápido, idéntico y sin fricciones en cualquier entorno de servidor.
- 2.3 Backend de Alto Rendimiento NET 9 Web API: Se implementó una API encargada de procesar de forma segura toda la lógica del negocio, validaciones de disponibilidad en tiempo real y la persistencia de datos en la base de datos en PostgreSQL través de Entity Framework Core. La API está completamente documentada e interactiva mediante Swagger, facilitando futuras integraciones o auditorías de código.
- 2.4 Frontend Interactivo y Eficiente Blazor: Utilizando la potencia de Blazor, se creó una interfaz de usuario moderna.
- 2.5 Esto permite al personal administrativo del turicentro gestionar las horas de entrada y salida, y el ingreso de visitantes de manera fluida, rápida y sin recargas molestas de página.
- 2.6 La base de datos relacional unifica el catálogo de cabañas, tarifas, registros de clientes y estados de ocupación, ofreciendo a la administración una fuente única para generar reportes confiables y agilizar la toma de decisiones.

Para la combinación de esta arquitectura se eligió pensando en la alta disponibilidad y escalabilidad, si en el futuro Hidden Valley decide expandirse o agregar facturación electrónica, se puede desarrollar como un microservicio extra sin tocar lo que ya esta programado.

3. RESUMEN DE SPRINTS

Linear: <https://linear.app/hospital-veterinario-patitas/team/PROYECT/all>

Link Drive con resumen de las Dailys:

https://drive.google.com/drive/folders/1VT_Kdbkl2G2Q5Er-uQtwZ0aQO3PcfFPn?usp=sharing

Durante el desarrollo del proyecto, el equipo adoptó Linear como herramienta principal de gestión de tareas y ciclos. Su implementación permitió estructurar el trabajo de manera ordenada y transparente, logrando una mejor coordinación entre los integrantes.

Cada tarea del proyecto se registró como un *issue* en Linear, con una descripción clara, responsable asignado y fecha estimada de entrega. Las tareas se agruparon según el área de trabajo (backend, frontend, base de datos, infraestructura), lo que permitió mantener una visión ordenada del avance. Linear permitió asignar niveles de prioridad (alta, media, baja), evitando que el equipo se dispersara en tareas secundarias y manteniendo el foco en lo más crítico.

Se configuraron *cycles* semanales, equivalentes a sprints, donde se planificaban las tareas a completar en ese periodo. Esto dio estructura al flujo de trabajo y permitió medir el progreso de manera constante.

3.1 DAILYS

Las reuniones rápidas de seguimiento se apoyaron directamente en Linear, siguiendo la metodología ágil:

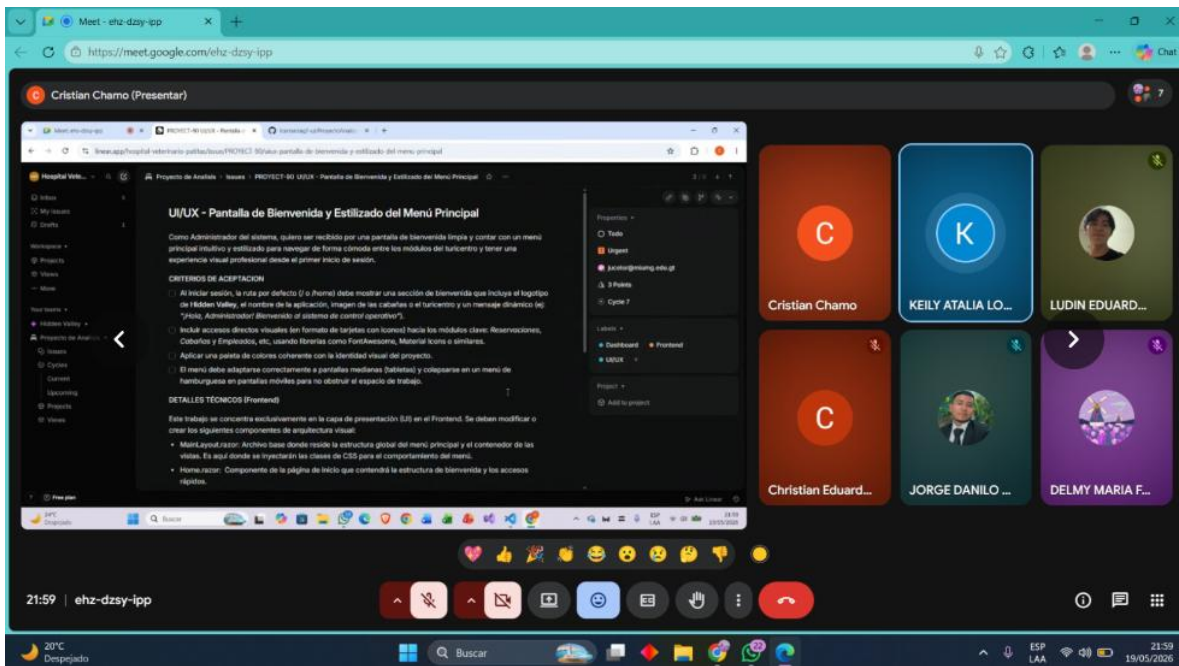
- **Revisión del tablero:** Cada miembro consultaba el estado de sus tareas en Linear antes de la reunión.
- **Informe breve:** Durante el daily, cada integrante respondía a tres preguntas clave:
 - Qué hice ayer (tareas completadas).
 - Qué haré hoy (tareas en curso o nuevas).
 - Qué obstáculos tengo (problemas o bloqueos).
- **Actualización en tiempo real:** Al finalizar el *daily*, se ajustaban las tareas en el tablero de Linear, manteniendo la información sincronizada y transparente para todos.

3.2 BENEFICIOS OBTENIDOS

Todos los miembros podían ver el estado de las tareas y ciclos en cualquier momento. Linear facilitó la comunicación entre backend, frontend y la base de datos y mantenimiento de infraestructura. Cada cambio quedaba registrado, lo que permitió auditar el progreso y justificar decisiones.

Las reuniones fueron cortas y enfocadas, gracias a que Linear servía como guía visual del avance. Las reuniones se llevaban a cabo mediante la aplicación Meet, por lo general se hacían alrededor de las 9:30 o 10:00 de la noche y tenían un promedio de 20 minutos cada daily, el tiempo variaba dependiendo de que se discutía en cada reunión.

Linear se convirtió en el eje de la organización del proyecto, ya que permitió estructurar las tareas en ciclos, dar claridad a los dailys y mantener un flujo de trabajo ágil y transparente.



linear.app/hospital-veterinario-patitas/team/PROYECT/all

Hospital Vete...

- Inbox 5
- My issues
- Reviews
- Drafts 1

Workspace

- Projects
- Views
- More

Your teams

- Hidden Valley
- Proyecto de Analisis
 - Issues
 - Cycles
 - Current

Issues

All issues Active Backlog

Done 32

PROJECT-90	UI/UX - Pantalla de Bienvenida y Estilizado del Menú Principal	Dashboard Frontend UI/UX	#29	7	3	May 18
PROJECT-91	Vista de Dashboard de Operaciones Diarias	Backend Dashboard Frontend	#34	7	5	May 20
PROJECT-89	Panel de Administración y Control de Reservasiones	Dashboard Frontend UI/UX	#28	7	3	May 18
PROJECT-88	Modal de Configuración y Gestión de Tipos de Cabaña	Frontend UI/UX	#38	8	5	May 18
PROJECT-87	Modal de Configuración y Gestión de Tipos de Servicio	Frontend UI/UX	#37	8	5	May 18
PROJECT-86	Vista Catálogo de Servicios Adicionales	Dashboard Frontend UI/UX	#32	7	5	May 18
PROJECT-85	Vista de Asignación y Control de Servicios en Reservasiones	Dashboard Frontend UI/UX	#35	7	5	May 18
PROJECT-84	Modulo para Pantalla de Inicio (Dashboard)	API Backend Base de Datos	#30	7	3	May 18
PROJECT-83	Modal de Puestos de Trabajo	Frontend UI/UX Validaciones	#36	7	5	May 18
PROJECT-82	Vista Panel de Administración y Control de Cabañas	Backend Dashboard Frontend	2 PRs	7	3	May 18
PROJECT-80	Adaptar RegistroReservasiones a la arquitectura corr...	API Backend Feature Validaciones	#23	6	5	May 12
PROJECT-76	API para ReservacionServicio	API Backend Base de Datos	#19	6	3	May 3
PROJECT-75	Modulo de Servicio	API Backend	#10	5	3	May 3

Linear Diffs
Review and merge code directly from Linear

https://linear.app/hospital-veterinario-patitas/issue/PROJECT-83/modal-de-puestos-de-trab...

Buscar

10:34 29/05/2026

4. APLICACION FINALIZADA

Link a GitHub: <https://github.com/lcarranzag2-ui/ProyectoAnalisisdeSistemas>

La aplicación engloba muy bien todo el esfuerzo del desarrollo (módulos, backend, frontend y bases de datos). El diseño del sistema prioriza la experiencia del usuario (UI/UX) mediante interfaces dinámicas e interactivas que facilitan la labor diaria del personal operativo sin sobrecargar las pantallas de datos complejos. Asimismo, el ecosistema técnico está estructurado con tecnologías de vanguardia bajo el framework .NET para el núcleo de servicios (backend) y Blazor para la interfaz de usuario (frontend), garantizando un rendimiento óptimo, escalabilidad y un manejo seguro de la concurrencia en la base de datos.

4.1 ARQUITECTURA TECNOLÓGICA

El sistema fue construido utilizando un enfoque de separación de responsabilidades con un stack tecnológico robusto:

- **Backend (API Rest):** Desarrollado sobre el ecosistema .NET (ASP.NET Core), encargado de exponer los servicios y procesar la lógica de negocio de forma segura.
- **Frontend:** Aplicación web interactiva construida con Blazor WebAssembly / Server, utilizando Bootstrap y Bootstrap Icons para lograr una interfaz de usuario adaptativa responsive, fluida y moderna.
- **Base de Datos:** Motor relacional robusto en PostgreSQL.

4.2 MÓDULOS PRINCIPALES DEL SISTEMA

Administración General: Mantenimiento y catálogos de entidades del negocio (Personas, Empleados, Clientes, Tipos de Servicio y Servicios individuales).

Panel de Reservaciones: Gestión completa del estado de las estadías (Recibida, Confirmada, Cancelada). Controla fechas de entrada y salida, asignación de cabañas, número de huéspedes y cálculo automático de montos totales.

Gestión de Cabañas: Panel de control para el inventario de habitaciones. Permite registrar especificaciones de capacidad, precios y actualizar estados operativos en tiempo real (Disponible, Ocupada, Mantenimiento, Reservada).

Servicios por Reservación: Administración y asignación de consumos extra vinculados de forma directa a las estadías activas. Permite acumular cantidades de servicios y eliminarlos dinámicamente mediante una vista modal centralizada.

Hidden Valley

Inicio

Dashboard

Personas

Reservaciones

Cabañas

Empleados

Cientes

Servicios

Reservacion Servicios

Hidden Valley

Sistema de Control Operativo

¡Hola, Administrador! Bienvenido al sistema de Hidden Valley

Personas

Reservaciones

Cabañas

Empleados

Cientes

Servicio

Reservacion Servicio

Dashboard

Hidden Valley

Inicio

Dashboard

Personas

Reservaciones

Cabañas

Empleados

Cientes

Servicios

Reservacion Servicios

Hidden Valley

Panel de Reservaciones

Gestión del ciclo de vida de reservaciones — Hidden Valley

Buscar por ID o nombre del cliente...

ID	Cliente	Cabaña (Tipo)	F. Entrada	F. Salida	Personas	Estado	Total	Acciones
#2	Cristian Eduardo Chamo Morales	Suite Matrimonial	01/06/2026	04/06/2026	2	Cancelada	450,00 €	Editar
#5	Cristian Eduardo Chamo Morales	Suite Matrimonial	27/05/2026	28/05/2026	2	Cancelada	150,00 €	Editar
#6	Cristian Eduardo Chamo Morales	Suite Matrimonial	27/05/2026	28/05/2026	2	Cancelada	150,00 €	Editar
#1	Juan José Pérez Méndez	Suite Matrimonial	24/05/2026	25/05/2026	2	Pagada	150,00 €	Editar
#4	Mario Estrada	Suite Matrimonial	23/05/2026	24/05/2026	2	Confirmada	220,00 €	Editar
#3	Cristian Eduardo Chamo Morales	Suite Matrimonial	21/05/2026	23/05/2026	1	Cancelada	300,00 €	Editar

Página 1 de 1 (6 registros)

Hidden Valley

Inicio

Dashboard

Personas

Reservaciones

Cabañas

Empleados

Cientes

Servicios

Reservacion Servicios

Hidden Valley

Gestión de Cabañas

Buscar por tipo o estado...

Buscar Cabañas

Gestionar Tipos de Cabaña

Registrar Nueva Cabaña

ID	Vista	Tipo de Cabaña	Capacidad	Precio Noche	Estado Actual	Acciones Administrativas
#1		Suite Matrimonial	2 personas	150.00	Ocupada	Ocupada Eliminar
#2		Suite Matrimonial	2 personas	150.00	Ocupada	Ocupada Eliminar
#3		Familiar Estándar	5 personas	250.00	Disponible	Disponible Eliminar
#10		Familiar Estándar	5 personas	250.00	Disponible	Disponible Eliminar

Mostrando página 1 de 1 (4 registros totales)

Anterior 1 Siguiente

Hidden Valley

Inicio

Dashboard

Personas

Reservaciones

Cabañas

Empleados

Cientes

Servicios

Reservacion Servicios

Hidden Valley

Operaciones del Dia

Resumen en tiempo real — Hidden Valley

Actualizar

Cabañas Disponibles2

Cabañas Ocupadas0

Personas Esperadas Hoy0

Proximas Reservaciones

No hay proximas reservaciones programadas

5. RESUMEN DE EJECUCIÓN DE PRUEBAS

Link a Drive de las pruebas:

https://drive.google.com/drive/folders/1NZwF5dxt1VzahuHRU_1PvUMSuhje5OGP?usp=drive_link

5.1 PRUEBAS UNITARIAS (UNIT TESTS)

Son pruebas automatizadas encargadas de aislar y analizar la porción más pequeña de código que contiene lógica dentro de la aplicación, como funciones, métodos o clases individuales. Se ejecutan de manera independiente, abstrayéndose por completo de cualquier factor externo como bases de datos, APIs o archivos del sistema, simulando estos componentes mediante objetos ficticios (Mocks).

- **¿Para qué las ocupamos en Hidden Valley?** En nuestro sistema las utilizamos para blindar los algoritmos y las reglas lógicas puras antes de que interactúen con los datos. Por ejemplo, validar que el sistema permita calcular bien el valor total de una reservación de cabaña o que pueda asignar de forma correcta servicios adicionales a una reservación ya existente.

5.2 PRUEBAS DE INTEGRACIÓN (INTEGRATION TESTS)

Son pruebas que evalúan cómo interactúan y se comunican múltiples componentes o módulos del software entre sí. A diferencia de las unitarias, aquí sí se pone a prueba el código junto con la infraestructura subyacente que utiliza la aplicación en la vida real, como bases de datos, sistemas de archivos o peticiones de red, habitualmente usando entornos controlados o simulados en memoria.

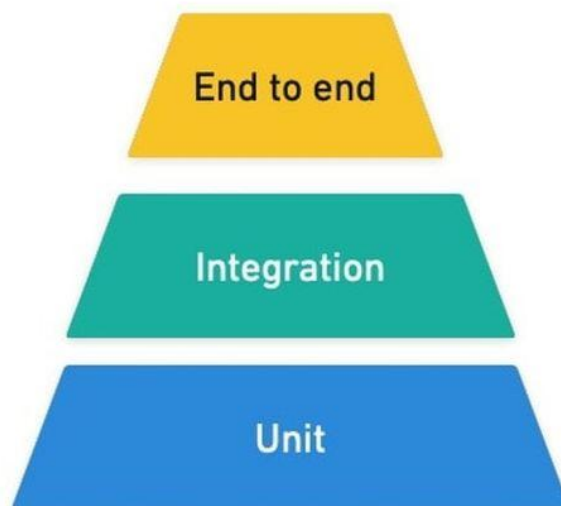
- **¿Para qué las ocupamos en Hidden Valley?** Las ocupamos principalmente para asegurar el correcto funcionamiento de nuestra API (HiddenValley.API). Con ellas validamos que los controladores respondan los códigos HTTP adecuados, que la base de datos respete las restricciones de integridad (por ejemplo, impedir que se borre un *Tipo de Cabaña* si aún existen *Cabañas* vinculadas a él), y que el *Dashboard* agrupe y calcule correctamente las métricas de ocupación en tiempo real basándose en los registros existentes.

5.3 PRUEBAS DE EXTREMO A EXTREMO (END-TO-END)

Son pruebas de alto nivel que simulan el comportamiento completo de un usuario real interactuando con la aplicación de principio a fin. Automatizan el uso del navegador web para dar clics, rellenar formularios y navegar por las pantallas, asegurando que toda la cadena del software (Interfaz de usuario, red, servidores, servicios externos y bases de datos reales) funcione en perfecta sincronía.

- **¿Para qué las ocupamos en Hidden Valley?** Las ocupamos de forma selectiva para garantizar el éxito de los flujos transaccionales y críticos del hotel, donde un fallo detendría la operación del negocio. Por ejemplo, las utilizamos para asegurar que un recepcionista pueda abrir el navegador, iniciar sesión en el frontend de Blazor, completar el formulario de una "Nueva Reservación", asignar cabañas y confirmar el registro, verificando visualmente que el sistema procese todo el circuito sin errores de comunicación entre el cliente y el servidor.

The Test Pyramid



CONCLUSION

El desarrollo del sistema para Hidden Valley consolida los objetivos académicos del curso de Análisis de Sistemas I, demostrando que el éxito de un software no radica únicamente en la escritura de código, sino en un riguroso proceso previo de diagnóstico, modelado y especificación de requerimientos. Al enfrentarse a las debilidades operativas del turicentro, el equipo logró traducir las necesidades del negocio (control de huéspedes, asignación de cabañas, optimización de limpieza y servicios adicionales) en un ecosistema digital integrado y modular. Esta experiencia de aprendizaje práctico no solo resuelve una problemática real de forma eficiente y escalable para Hidden Valley, sino que nos ayuda a comprender a nosotros como estudiantes en el uso de herramientas de vanguardia, flujos de trabajo ágiles y criterios arquitectónicos esenciales para su futuro desempeño profesional en el desarrollo de software.